

# Tessera

*Gestao de clubes e bilheteira digital*

---

APRESENTACAO DE PROGRESSO

## **Grupo 31**

Bruno Pinto, n.o 48064

Daniel Cabaca, n.o 48070

*Orientador: Jose Faustino*

# O que é o Tesserá?

Motivação, contexto e visão geral da solução

## O PROBLEMA

**Clubes de futebol de divisões inferiores em Portugal enfrentam desafios operacionais por falta de ferramentas digitais.**

- \* Bilhetes em papel — custos de impressão, risco de falsificação, dificuldade no controlo de entradas
- \* Ficha técnica preenchida à mão — erros, atrasos e dificuldade de consulta
- \* Sem rastreio de vendas, sem histórico digital, sem análise de dados

## A SOLUÇÃO

**Plataforma digital para gestão completa de bilheteira e ficha técnica dos jogos.**

- \* Aplicação web (React SPA) para administradores e adeptos
- \* Aplicação Android para validação de bilhetes e ficha técnica
- \* Bilhetes digitais com QR code (MB WAY, multibanco, dinheiro)
- \* Registo de ocorrências do jogo em tempo real
- \* Histórico digital de jogos e relatórios de bilheteira

**Caso de uso real:** *SU 1.º Dezembro — clube parceiro disponível para cooperar na elaboração da solução*

# Requisitos & Perfis de utilizador

*Tres perfis distintos com necessidades diferentes*

ADM

## Administrador

- Gestao de clubes e equipas
- Gestao de jogadores e planteis
- Criacao de jogos e eventos
- Configuracao de bilheteira
- Consulta de relatorios e estatisticas

STF

## Staff

- Validacao de bilhetes (QR code)
- Preenchimento da ficha tecnica
- Registo de ocorrencias em tempo real (golos, cartoes, substituicoes)
- Operacoes no dia de jogo

FAN

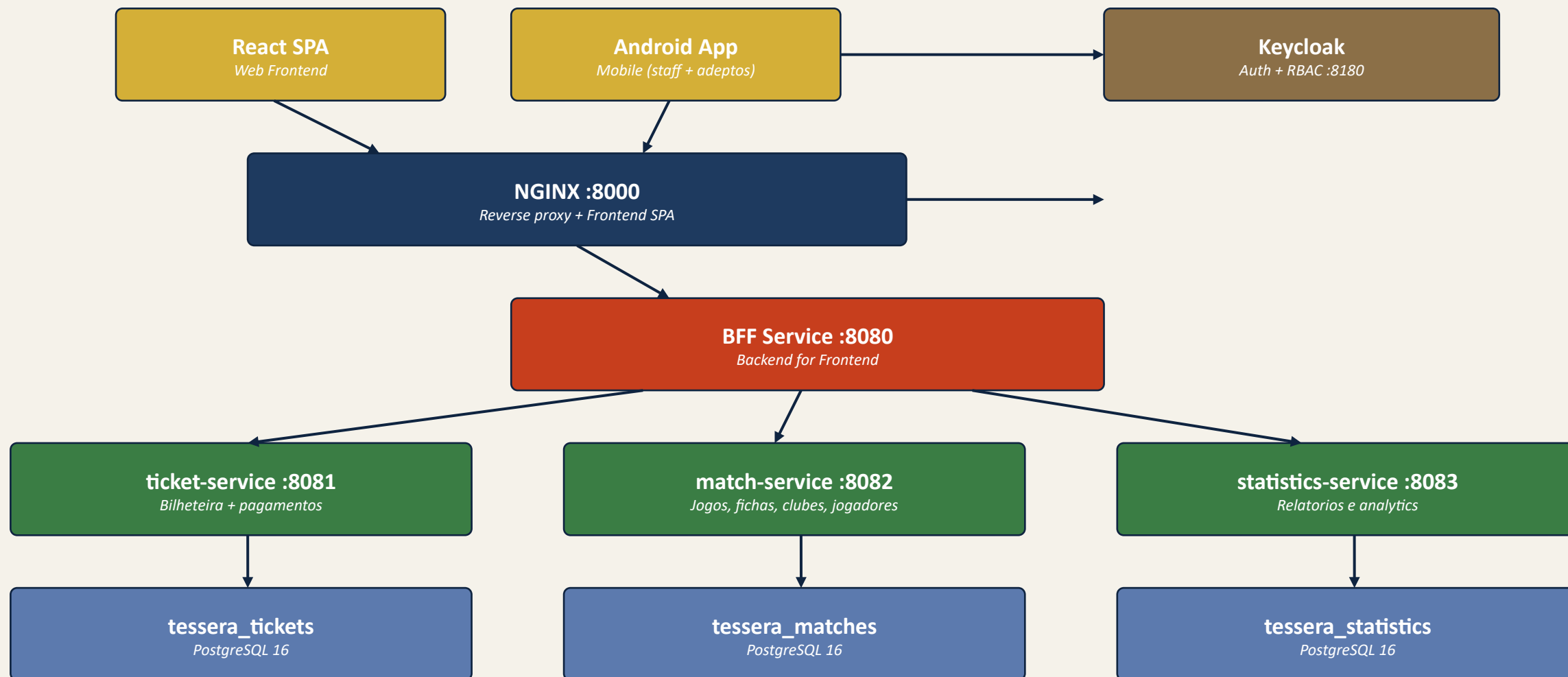
## Adepto

- Compra de bilhetes digitais
- Pagamento via MB WAY
- Visualizacao de bilhete (QR)
- Consulta de jogos e resultados
- Acesso a fichas tecnicas

**Requisitos nao funcionais:** Desempenho (validacao rapida) - Seguranca (RBAC + HTTPS) - Disponibilidade - Usabilidade

# Arquitetura do sistema

Microservicos, BFF e isolamento de dados



Cada servico com BD propria, deploy independente, comunicacao via REST sincrono entre BFF e servicos internos.

# Microsservicos

*Responsabilidades e dominio de cada servico*

## BFF Service

:8080 (sem BD)

Backend for Frontend

Ponto de entrada unico para a API. Adapta respostas a cada cliente, agrega chamadas a multiplos servicos.

## Ticket Service

:8081 tessera\_tickets

Bilheteira

Eventos, compra de bilhetes (PENDING -> PAID -> VALIDATED), validacao por QR code, pagamentos.

## Match Service

:8082 tessera\_matches

Atividade desportiva

Clubes, equipas, jogadores, jogos, fichas tecnicas e ocorrencias (golos, cartoes, substituicoes).

## Statistics Service

:8083 tessera\_statistics

Analise e relatorios

Servico de leitura. Relatorios de bilheteira, estatisticas de jogos e jogadores, historico.

**Decisao:** *Clubes, equipas e jogadores vivem dentro do match-service (em vez de servico separado) — sao consumidos quase exclusivamente pela ficha tecnica.*

# Stack tecnologico

Tecnologias selecionadas para cada componente do sistema

|               |                            |                |
|---------------|----------------------------|----------------|
| BACKEND       | Spring Boot + Kotlin       | 3.4.4 / 1.9.25 |
| MOBILE        | Android (Kotlin + Compose) | —              |
| MIGRACOES BD  | Flyway                     | —              |
| REVERSE PROXY | NGINX (Alpine)             | —              |
| BUILD TOOL    | Gradle (Kotlin DSL)        | 8.13           |
| API SPEC      | OpenAPI 3.1 + Redocly      | —              |

|               |                           |          |
|---------------|---------------------------|----------|
| FRONTEND WEB  | React + TypeScript + Vite | —        |
| BASE DE DADOS | PostgreSQL (Alpine)       | 16       |
| AUTENTICACAO  | Keycloak (OIDC + OAuth2)  | 26.0     |
| CONTAINERS    | Docker + Docker Compose   | —        |
| JDK           | Eclipse Temurin           | 21       |
| PAGAMENTOS    | API SIBS / MB WAY         | planeado |

# O que esta feito - Infraestrutura

*Setup completo de Docker, NGINX e Keycloak*

OK

## Docker Compose

9 containers orquestrados — 4 servicos backend, 3 BDs, NGINX, Keycloak. Rede interna com DNS, volumes persistentes, health checks.

OK

## NGINX como reverse proxy

Routing /api/\* -> BFF, /realms/\* + /admin/\* -> Keycloak, /\* -> React SPA. Frontend embutido na imagem (multi-stage build).

OK

## Keycloak — autenticao e autorizacao

Realm 'tessera' com 3 roles (admin, staff, fan), 3 clients (web, android, bff) com PKCE, 3 utilizadores de teste, importacao automatica.

OK

## Scripts de automacao (PowerShell)

buildAll.ps1, start.ps1, stop.ps1, reset.ps1 e scripts individuais por servico (reset\_xxx\_service.ps1) para builds e gestao de containers.

OK

## Documentacao

6 documentos Markdown — architecture, docker, nginx, keycloak, scripts, getting-started — atualizados a medida que o projeto evolui.

# O que esta feito - Backend

Migracao de monolito para microservicos e proxy BFF

## PONTO DE PARTIDA

### Backend monolitico

*Spring Boot + Kotlin*

Apenas 2 entidades:

- \* Event
- \* Ticket

Endpoints basicos:

- \* POST /api/tickets
- \* POST /api/tickets/validate

## ARQUITETURA ATUAL

### bff-service

*Proxy + agregacao*

:8080

### ticket-service

*Bilheteira (event, ticket)*

:8081

### match-service

*Esqueleto (a desenvolver)*

:8082

### statistics-service

*Esqueleto (a desenvolver)*

:8083

**Cada service:** BD propria + Flyway migrations + Dockerfile + build script + reset script



# O que esta feito - Especificacao da API

*Contrato OpenAPI 3.1 como fonte de verdade*

## Abordagem contract-first

*A especificacao OpenAPI e o contrato vinculativo entre os 3 consumidores (web, android, BFF) e a backend. A implementacao deriva da spec.*

1 Definir endpoints e schemas em YAML (docs/api/)

2 Validar com Spectral (linter de OpenAPI)

3 Gerar documentacao HTML interativa (Redocly)

4 Servir Swagger UI no BFF (/api/docs)

5 Gerar clientes tipados (TypeScript + Kotlin)

## RECURSOS DA API

### Match-service

- \* /clubs - clubes
- \* /teams - equipas
- \* /players - jogadores
- \* /venues - estadios
- \* /matches - jogos
- \* /matches/{id}/sheet - ficha tecnica

### Ticket-service

- \* /tickets - bilhetes
- \* /events - eventos de bilheteira
- \* /payments - pagamentos

### Statistics-service

- \* /stats/matches/{id}
- \* /stats/players/{id}
- \* /stats/teams/{id}

**Convencoes:** /api/v1 - camelCase - ISO 8601 - Problem Details (RFC 7807) - JWT Bearer (Keycloak)

# Modelo de dados

Entidades principais por contexto (BD por servico)

## MATCH-SERVICE

### Club

*id, name, foundedYear, crestUrl*

### Team

*id, clubId, category*

### Player

*id, teamId, name, position, shirtNumber*

### Venue

*id, name, capacity, address*

### Match

*id, homeTeamId, awayTeamId, kickoffAt, status*

### MatchSheet

*id, matchId, locked*

### Occurrence

*id, minute, type (GOAL/CARD/SUB)*

## TICKET-SERVICE

### Event

*id, matchId, name, priceNormal, priceSupporter, status*

### Ticket

*id, eventId, code (UUID), price, status*

### Lifecycle:

#### PENDING

*(criado, aguarda pagamento)*

#### PAID

*(pagamento confirmado)*

#### VALIDATED

*(usado a entrada)*

## STATISTICS-SERVICE

### MatchStats

*matchId, scorers, cards, ...*

### PlayerStats

*playerId, season, goals, cards*

### TeamStats

*teamId, season, won, drawn, lost*

### EventSales

*eventId, sold, validated, revenue*

### Tabelas read-only

*alimentadas via*

### RabbitMQ events

*(planeado)*

# Demonstracao

*Sistema funcional end-to-end*

## Fluxo completo a correr localmente

```
> .\scripts\run\start.ps1

[OK] Building all services... (buildAll.ps1)
bff-service      : SUCCESS
ticket-service   : SUCCESS
match-service    : SUCCESS
statistics-service : SUCCESS

[OK] Starting Docker containers...
9 containers running on tessera-net

All services are up!
App:      http://localhost:8000
BFF API:   http://localhost:8000/api
Keycloak:  http://localhost:8180
```

## Demonstracao em vivo:

- \* Frontend React acessivel em <http://localhost:8000>
- \* Login via Keycloak com 3 utilizadores diferentes (admin, staff, adepto) — JWT contem roles
- \* Compra de bilhete: SPA -> NGINX -> BFF -> ticket-service -> PostgreSQL
- \* Validacao de bilhete por UUID — alteracao de estado PAID -> VALIDATED
- \* Consola de admin do Keycloak para gestao de utilizadores e roles

# Desafios tecnicos enfrentados

*Problemas reais e como foram resolvidos*

## **Problema:** Build do gradlew falhava em Alpine Linux

**Causa:** O BusyBox sh nao interpreta corretamente os JVM args do gradlew (DEFAULT\_JVM\_OPTS com quotes aninhadas)

**Solucao:** Adotamos abordagem runtime-only: build do JAR na maquina local, Dockerfile apenas COPY do JAR pre-compilado

## **Problema:** NGINX retornava 404 em /api/ e /realms/

**Causa:** A imagem base do NGINX inclui um default.conf com server block na porta 80 que intercetava os pedidos antes da nossa configuracao

**Solucao:** Adicionamos RUN rm /etc/nginx/conf.d/default.conf no Dockerfile

## **Problema:** Porta 80 ocupada no host (outro NGINX em WSL)

**Causa:** Maquina de desenvolvimento ja tinha um NGINX a correr na porta 80

**Solucao:** Configuramos o nosso NGINX para a porta 8000

## **Problema:** Keycloak 26 sem prefixo /auth/

**Causa:** Ao contrario das versoes < 25, o Keycloak 26 serve diretamente em /realms/, /admin/, /resources/

**Solucao:** Atualizamos a configuracao NGINX com locations explicitos para cada caminho do Keycloak

## **Problema:** CORS no browser ao chamar a API

**Causa:** Frontend chamava http://localhost:8080 diretamente, gerando cross-origin request

**Solucao:** Mudamos o frontend para path relativo /api — todos os pedidos passam pelo NGINX

# O que falta fazer

*Funcionalidades em falta e proximos passos*

## BACKEND

- \* Implementar match-service (CRUD de clubes, equipas, jogadores, jogos)
- \* Implementar ficha tecnica e ocorrencias
- \* Implementar statistics-service
- \* Integracao SIBS / MB WAY no ticket-service
- \* Eventos assincronos com RabbitMQ
- \* Validacao de tokens JWT no BFF
- \* Testes (unitarios + integracao)

## FRONTEND + MOBILE

- \* SPA React: ecras de gestao para admin
- \* SPA React: compra de bilhetes para adeptos
- \* Geracao de cliente TypeScript a partir do OpenAPI
- \* App Android: validacao de QR (staff)
- \* App Android: ficha tecnica em tempo real
- \* App Android: visualizacao de bilhetes (adeptos)
- \* OCR de fichas tecnicas em papel (opcional)

## OPS + DELIVERY

- \* Configurar HTTPS (certificado SSL)
- \* Pipeline CI/CD
- \* Substituir BD H2 do Keycloak por PostgreSQL
- \* Logs centralizados
- \* Deploy em ambiente de producao
- \* Documentacao de utilizador final
- \* Validacao com SU 1.o Dezembro

# Proximos passos & Planeamento

---

## FASE 1

**Implementacao backend (match-service)** - Clubes, equipas, jogadores, jogos. Migracoes Flyway, DTOs, controllers.

## FASE 2

**Bilheteira completa + integracao SIBS** - Pagamentos MB WAY, validacao JWT, codigos QR.

## FASE 3

**Ficha tecnica em tempo real** - Endpoints de match sheet, lineups, ocorrencias.

## FASE 4

**Frontend e Android** - Ecras de utilizador, geracao de clientes API, app Android funcional.

## FASE 5

**Statistics + RabbitMQ + Deploy** - Eventos assincronos, relatorios, deploy em producao com SU 1.o Dezembro.

# Perguntas?

Obrigado pela atencao - Bruno Pinto & Daniel Cabaca - Grupo 31

[github.com/bruno-pinto-git/tessera](https://github.com/bruno-pinto-git/tessera)